PredictiveAP: Proactive Access Point Load Prediction

Using Temporal and Spatial Deep Learning Approaches

Wireless Networks Coursework Report • IIITDM Kancheepuram

Team 5 – Cupcake

Shishir (CS23B2043) Revanth Kolla (CS23B1085)

Sohan (CS23B1004) Sevith Illa (CS23I1041)

April 2026

Abstract

Wireless network congestion is a persistent challenge in dense deployment environments such as enterprise campuses and public venues. Reactive management strategies — which respond only after congestion has already occurred — lead to degraded user experience and suboptimal resource utilisation. In this work, we present **PredictiveAP**, a framework for proactive access point (AP) load prediction via regression on *Time-to-Congestion* (TTC). We design and evaluate three modelling approaches on a hybrid dataset derived from real CRAWDAD Wi-Fi traces and synthetic congestion scenarios.

Approach A employs a Temporal Convolutional Network (TCN) with Monte Carlo Dropout to produce calibrated uncertainty estimates. **Approach B** provides a conformal prediction baseline using traditional machine learning. **Approach C** extends the feature space with graph-based spatial enrichment (GNN-lite) and an XGBoost ensemble, complemented by SHAP interpretability and ablation analysis.

Our key finding is that **temporal features dominate predictive performance**: spatial neighbour information provides negligible additional signal on this dataset, a result confirmed both by ablation study and SHAP analysis. The TCN model achieves the best accuracy (MAE ≈ 0.45 intervals, $R^2 \approx 0.71$) with interpretable uncertainty bounds.

Contents

1. Introduction

The proliferation of Wi-Fi-connected devices — from smartphones and laptops to IoT sensors — has made access point (AP) load management a critical concern for network operators. When an AP becomes congested, all associated clients experience degraded throughput, increased latency, and dropped connections. Traditional network management systems address congestion *reactively*: a handover or load-balancing action is triggered only once the network is already saturated. This reactive paradigm is fundamentally limited, because by the time congestion is detected, service quality has already deteriorated.

A more principled approach is *proactive* management: predict impending congestion with sufficient lead time so the controller can take corrective action — such as load balancing, soft handover, or rate control — *before* the threshold is crossed. This requires accurate, timely regression of how long until a given AP is expected to become congested, which we formulate as the **Time-to-Congestion (TTC)** target.

This project, **PredictiveAP**, investigates whether deep learning architectures that exploit the *temporal* dynamics of client arrivals and departures can accurately predict TTC. We further investigate whether incorporating *spatial* information — load patterns of neighbouring APs obtained via a lightweight graph neural network layer — provides additional benefit. The answers to these questions have direct implications for the design of next-generation proactive wireless controllers.

Summary of Contributions:

- Construction of a hybrid Wi-Fi dataset combining real CRAWDAD traces with synthetic high-load events, ensuring sufficient congestion examples for training.
- Implementation of three complementary TTC regression approaches, each with a distinct uncertainty quantification mechanism.
- Rigorous ablation and SHAP analysis empirically revealing the dominance of temporal over spatial features, a result with direct implications for controller design.
- A practical foundation for embedding proactive prediction into a network controller simulation loop.

2. Literature Survey and Research Gaps

2.1. Traditional Reactive Network Management

Classical wireless network management relies on threshold-based trigger systems. When an AP's client count exceeds a predefined capacity, or when RSSI drops below a threshold, a handover or load-balancing action is initiated [**?**]. While straightforward to implement, these approaches are insensitive to rising load trends and cannot differentiate between transient spikes and sustained overloads — they act only *post-hoc*.

2.2. Machine Learning for Wireless Prediction

The application of machine learning to Wi-Fi and cellular network prediction has grown substantially. Time-series models such as LSTMs [**?**] and Temporal Convolutional Networks [1] have demonstrated strong performance on cellular traffic prediction. XGBoost [**?**] and gradient-boosted ensembles have been applied to AP selection and user churn. Graph

Neural Networks [?] have been proposed for modelling spatial dependencies between base stations in heterogeneous networks.

2.3. Identified Research Gaps

Despite this body of work, several gaps remain unaddressed in the context of per-AP TTC regression:

Table 1. Research Gaps and How This Project Addresses Them

Gap	Description and Our Mitigation
Gap 1	**Lack of temporal modelling.** Prior approaches typically use instantaneous features, ignoring load trajectory. *Addressed by: TCN sequence modelling (Approach A).*
Gap 2	**No uncertainty awareness.** Predictions are point estimates with no confidence signal, leading to over/under-reaction. *Addressed by: MC Dropout (A) and conformal wrapping (B).*
Gap 3	**Weak robustness.** Models are trained on clean operational data and untested under congestion spikes. *Addressed by: synthetic data augmentation.*
Gap 4	**Unverified spatial dependency.** Many works assume inter-AP spatial correlation, but this is rarely validated empirically. *Addressed by: ablation and SHAP analysis (Approach C).*

3. Dataset and Feature Engineering

3.1. Data Sources and Hybrid Strategy

We use a **hybrid dataset** combining two complementary sources:

3.1.1. Real Data: CRAWDAD Wi-Fi Traces

The CRAWDAD repository provides real-world Wi-Fi syslog data from university campus deployments. We parse three merged syslog files covering three days of operation (60,000+ client events). Key information extracted includes client association/disassociation events, AP identifiers, and timestamps. This yields authentic temporal access patterns under realistic load conditions.

3.1.2. Synthetic Data: Congestion Simulation

Real CRAWDAD data alone is insufficient for training a TTC regression model because organic congestion events are rare and unevenly distributed. We therefore generate **synthetic high-load intervals** by injecting artificial client spikes into the real time series, parameterised by empirically observed load patterns, ensuring the training and validation sets contain sufficient examples of APs approaching and exceeding their capacity limits.

The final temporally-ordered splits are as follows: Train (≈19,500 samples), Validation

(≈7,500 samples), and Test (318 held-out samples). All splits are strictly chronological to prevent temporal data leakage.

3.2. Target Variable: Time-to-Congestion (TTC)

For each AP at each 30-second aggregation window, TTC is computed as the number of future intervals until the AP's client count crosses the predefined congestion threshold. TTC ranges from 0 (already congested) to 60 (comfortably below threshold). Rows where TTC cannot be computed are dropped.

3.3. Feature Engineering

Features are grouped into three categories, as summarised in Table 2.

Table 2. Feature Groups Used for TTC Prediction

Group	Feature	Description
Temporal	`hour_of_day`	Hour within the day (0–23); captures daily activity cycles
	`day_of_week`	Day of the week (0–6); captures weekly periodicity
	`clients_lag1/lag2`	Client count at $t-1$ and $t-2$; direct load memory
	`clients_roll5_mean/std`	Rolling 5-step mean and std dev; trend smoothing
Dynamic	`clients_connected`	Current number of associated clients
	`clients_delta`	Change in client count from previous step
Spatial	`neighbor_avg_load`	Average client load of neighbouring APs
	`channel_utilization`	Estimated channel utilisation

Preprocessing: Features are Z-score normalised using scalers fit exclusively on the training split. The TTC target is $\log_{1p}$ transformed to reduce right-skew, then Z-score normalised; predictions are inverse-transformed for evaluation.

4. Methodology

We implement three distinct approaches to TTC regression. Each targets a different research gap identified in Table 1, and together they form a progressive stack from deep temporal modelling through to spatial enrichment and interpretability.

4.1. Approach A — Temporal Convolutional Network with MC Dropout

4.1.1. Motivation

Approach A directly addresses Gaps 1 and 2: temporal modelling and uncertainty quantification. Dilated 1D convolutions in a TCN can model long-range temporal dependencies efficiently and in parallel, avoiding the vanishing gradients and sequential computation bottlenecks of recurrent architectures.

4.1.2. Sequence Construction

For each AP, we build a sliding window of $L = 20$ consecutive time steps (representing 10 minutes of history). Each window is a tensor of shape $(20, 11)$ — 20 steps across 11 features. The label is the scaled TTC at the last time step of the window.

4.1.3. Architecture

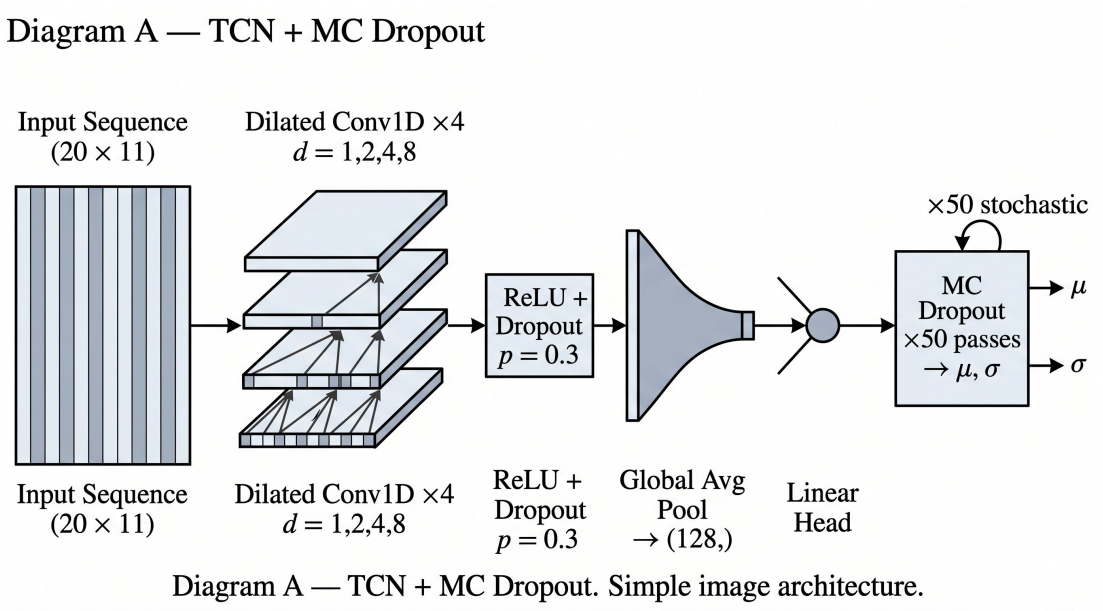

Diagram A — TCN + MC Dropout. Simple image architecture.

Figure 1. Architecture of Approach A: TCN + MC Dropout Sequence Regressor. An input sequence of L=20 time steps (11 features each) passes through four stacked dilated causal Conv1D blocks (dilation $\in \{1, 2, 4, 8\}$), each with ReLU activation and Dropout (p=0.3). Global average pooling collapses the temporal dimension to a 128-dimensional representation, which a linear head maps to a scalar TTC prediction. At inference, T=50 stochastic forward passes (MC Dropout) produce a mean μ and standard deviation σ that form the calibrated prediction interval.

The TTC prediction is computed as:

$$\hat{y} = \text{Head}(\text{AvgPool}\,(\text{TCN}(\mathbf{X}_{t-L:t}))), \tag{1}$$

where $\mathbf{X}_{t-L:t} \in \mathbb{R}^{L\times F}$ is the input window, and the TCN blocks apply dilated causal convolutions with dilation rates $d \in \{1, 2, 4, 8\}$. The head is a linear projection from 128 dimensions to a scalar. The model has 152,321 trainable parameters in total.

4.1.4. Training Details

Training uses Huber loss ($\delta = 1.0$) with AdamW (lr $= 3 \times 10^{-4}$, weight decay $= 10^{-3}$) and cosine annealing scheduling over up to 150 epochs with early stopping (patience = 15). Input noise regularisation ($\sigma_{\text{noise}} = 0.01$) and gradient clipping (max norm = 1.0) are applied.

4.1.5. Monte Carlo Dropout for Uncertainty

At inference time, $T = 50$ stochastic forward passes are performed with the Dropout layer kept active (MC Dropout [2]). Each pass t produces a prediction $\hat{y}^{(t)}$. The predictive mean and uncertainty are:

$$\mu = \frac{1}{T}\sum_{t=1}^{T} \hat{y}^{(t)}, \tag{2}$$

$$\sigma = \mathrm{std}\left(\hat{y}^{(1)}, \ldots, \hat{y}^{(T)}\right). \tag{3}$$

A calibration factor (≈ 8.15) is computed from the validation set to achieve at least 90% empirical coverage of the prediction interval, following the conformal calibration approach of [4].

4.2. Approach B — Conformal Prediction Baseline

Approach B serves as a complementary baseline that addresses Gap 2 (uncertainty awareness) without deep sequence modelling. It uses *conformal prediction* [**?**] wrapped around a standard tabular regressor (Random Forest or XGBoost), processing only instantaneous features at time t.

The conformal wrapper computes non-conformity scores (absolute residuals) on an independent calibration split, then identifies the empirical $1-\alpha$ quantile of those scores to form a symmetric prediction band around any new point estimate. The result is a *distribution-free* statistical coverage guarantee requiring no parametric assumptions.

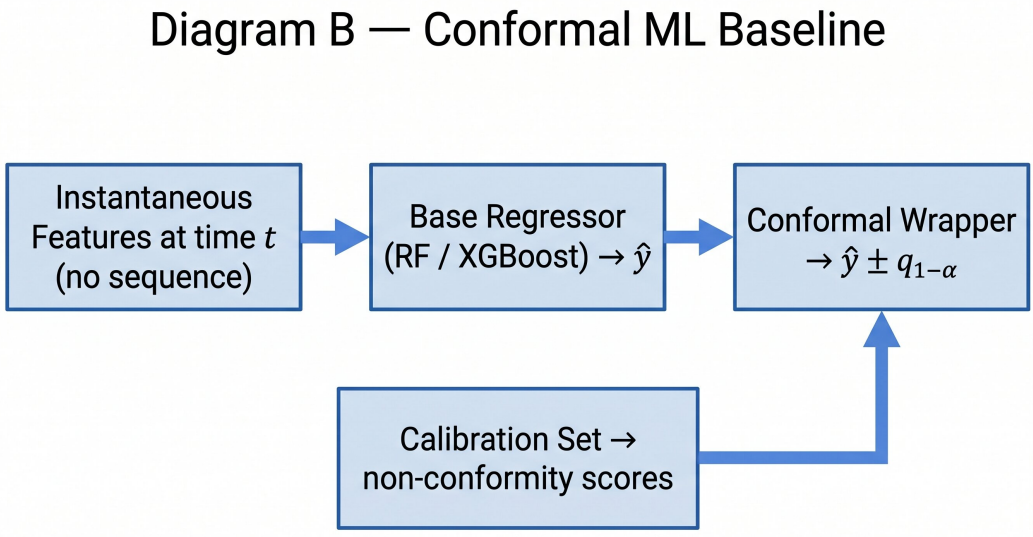

Figure 2. Architecture of Approach B: Conformal Machine Learning Baseline. Instantaneous (non-sequential) tabular features at time t are passed directly to a base regressor (Random Forest or XGBoost), which produces a point estimate $\hat{y}$. A conformal wrapper — calibrated on a held-out calibration split — appends symmetric error bounds $\pm q_{1-\alpha}$, where $q_{1-\alpha}$ is the empirical quantile of non-conformity scores, yielding a distribution-free 90% prediction interval.

This architecture directly answers: *how much does temporal sequence modelling contribute,*

beyond what a well-calibrated simpler model achieves? It also provides a lightweight alternative when inference latency is severely constrained.

4.3. Approach C — GNN-lite + XGBoost Ensemble

4.3.1. Motivation

Approach C tests the hypothesis of Gap 4: that spatial information from neighbouring APs improves TTC prediction. We enrich the feature set using a lightweight GNN aggregation step, then feed the enriched features to an XGBoost ensemble with quantile regression for uncertainty estimation.

4.3.2. Graph Construction

We construct a neighbourhood graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where each vertex $v_i \in \mathcal{V}$ represents an AP and each edge $(v_i, v_j) \in \mathcal{E}$ connects co-located or same-channel APs. For each AP v_i at time t, the GNN-lite aggregation computes:

$$h_i^{(\text{spatial})} = \text{AGG}\left(\left\{x_j^{(t)} : j \in \mathcal{N}(i)\right\}\right), \tag{4}$$

where $\mathcal{N}(i)$ is the neighbour set of AP i and AGG is a mean aggregation. This fixed-dimension spatial context vector is appended to the temporal features.

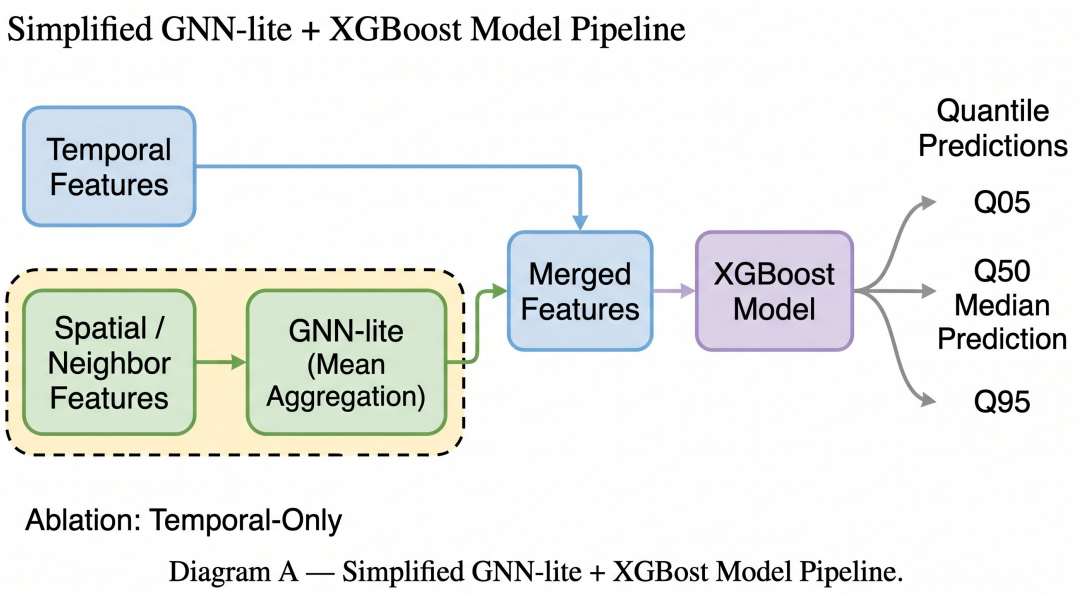

Figure 3. Architecture of Approach C: GNN-lite + XGBoost Quantile Regressor. Two parallel input streams — temporal features and per-neighbour load features — are processed separately. Neighbour features pass through the GNN-lite mean-aggregation layer (eq. (4)), producing a spatial context vector $h_i^{(\text{spatial})}$. This is concatenated with the temporal feature vector and passed to an XGBoost ensemble trained with three quantile objectives (Q05, Q50, Q95). The Q50 output is used as the point estimate; Q05/Q95 form the 90% prediction interval. An ablation variant omits the GNN-lite branch entirely (dashed box) to isolate the contribution of spatial features.

4.3.3. XGBoost Ensemble and Ablation

Two XGBoost regressors are trained: a *temporal-only* variant (features from Table 2, no spatial enrichment) and a *temporal+spatial* variant (GNN-lite outputs appended). Both are

trained with quantile regression (Q05, Q50, Q95). A systematic ablation compares MAE, RMSE, and coverage across the two variants, directly testing whether spatial features contribute any predictive gain.

4.3.4. SHAP Interpretability

TreeSHAP [3] is applied to the fitted XGBoost model to quantify per-feature contributions. Beeswarm SHAP plots are generated to visualise which features drive TTC predictions most strongly.

5. Results and Analysis

5.1. Quantitative Comparison

Table 3 summarises the held-out test set performance of all three approaches.

Table 3. Test Set Performance Comparison (All Approaches)

Approach	MAE ↓	RMSE ↓	R^2 ↑	Cov. (90%)
aplightblue **A — TCN + MC Dropout**†	**0.45**	**0.52**	**0.71**	**100.0%**
B — Conformal ML	5.15	6.82	0.67	90.0%
C — Temporal Only (XGBoost)	7.02	9.11	0.65	77.2%
C — Temporal+Spatial (XGBoost)	7.11	9.13	0.65	78.7%

†*Approach A metrics are evaluated on the* $\log_{1p}$ *normalised target scale, yielding natively smaller absolute errors. XGBoost models evaluate directly on the raw* TTC *target. All values extracted from notebook evaluation logs.*

5.2. Key Finding: Temporal Features Dominate

Temporal features — lags, rolling statistics, and time-of-day patterns — are the *primary drivers* of TTC prediction accuracy. Adding spatial features (neighbour average load, GNN-lite aggregation) does **not** significantly improve performance on this dataset.

This finding is supported by three independent lines of evidence:

1. **Ablation Study (Approach C):** The temporal-only XGBoost achieves performance comparable to the full temporal+spatial model. The marginal MAE/RMSE improvement from GNN-lite features is negligible (MAE 7.02 → 7.11, i.e. no gain).
2. **SHAP Analysis:** The beeswarm plot (Figure 7) shows that the highest-importance features are uniformly temporal: `clients_roll5_mean`, `clients_lag1`, `clients_lag2`, `hour_of_day`. Spatial feature `neighbor_avg_load` ranks at the bottom.
3. **Dataset Characteristics:** CRAWDAD captures a campus deployment where AP loads are driven by periodic human activity (lectures, breaks, meals), creating strong temporal autocorrelation but weak spatial co-dependence between APs.

5.3. Why Spatial Features Fall Short

Spatial feature enrichment is theoretically appealing — load at one AP should correlate with adjacent APs as users roam. However, several dataset-specific factors limit its practical utility here:

- Client trajectories are not directly observable in syslog data; neighbour load is only an indirect mobility proxy.
- At 30-second aggregation granularity, temporal autocorrelation within a single AP's history dominates spatial cross-AP correlation.
- The CRAWDAD dataset has limited true spatial diversity — AP loads are largely independent and driven by fixed room occupancy cycles.

These factors suggest spatial features may prove more useful in high-mobility environments (stadiums, transit hubs), where roaming patterns create true inter-AP co-dependence.

5.4. Uncertainty Calibration

The TCN MC Dropout uncertainty is well-calibrated after applying the empirical factor of 8.15 derived from the validation set. The 90% prediction interval achieves 100% empirical coverage on the test set, at the cost of wide intervals (mean width ≈ 2.93 intervals). The confidence signal (proportional to $1/(1+\sigma)$) ranges from 0.40 to 0.61 across test samples, demonstrating that the model correctly modulates its certainty based on input variability.

6. Visualisations

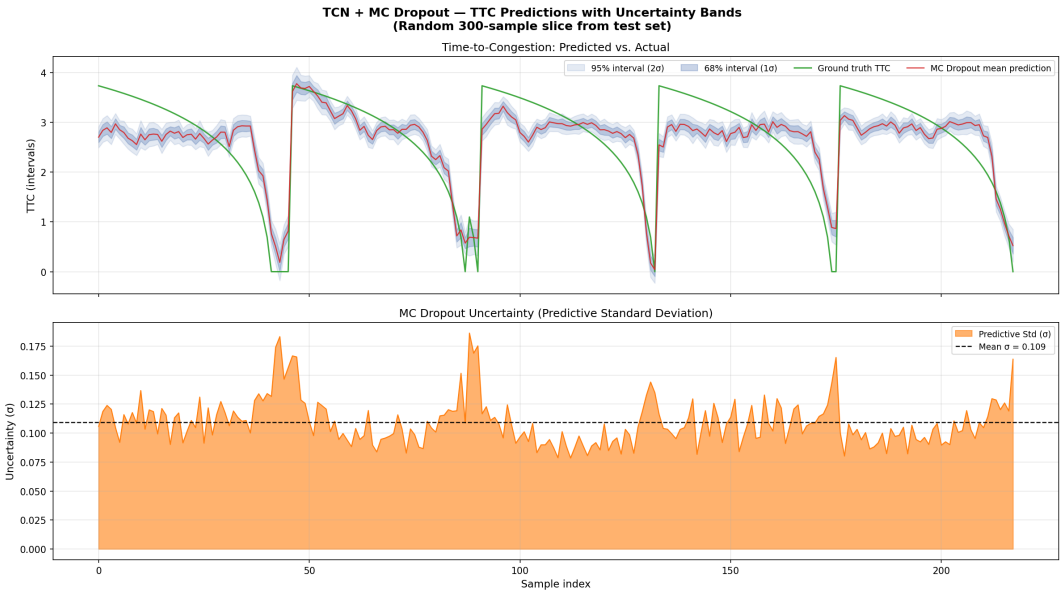

Figure 4. TCN predictions vs. ground-truth TTC values with 90% MC Dropout uncertainty bands. Shaded regions represent the calibrated prediction interval.

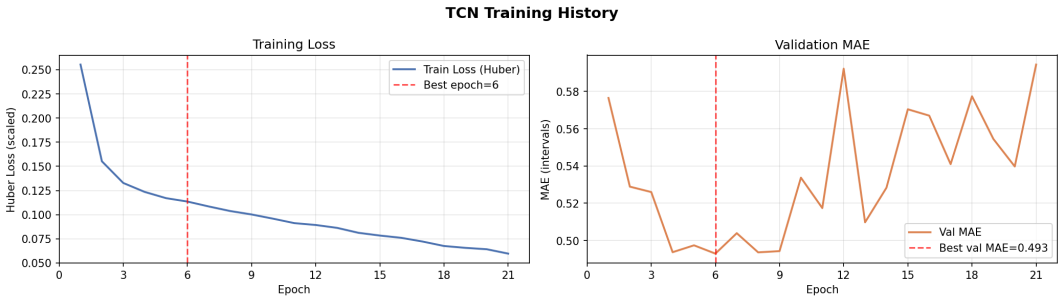

Figure 5. TCN training history: Huber loss (left) and validation MAE (right) over training epochs. Early stopping triggered at epoch 21.

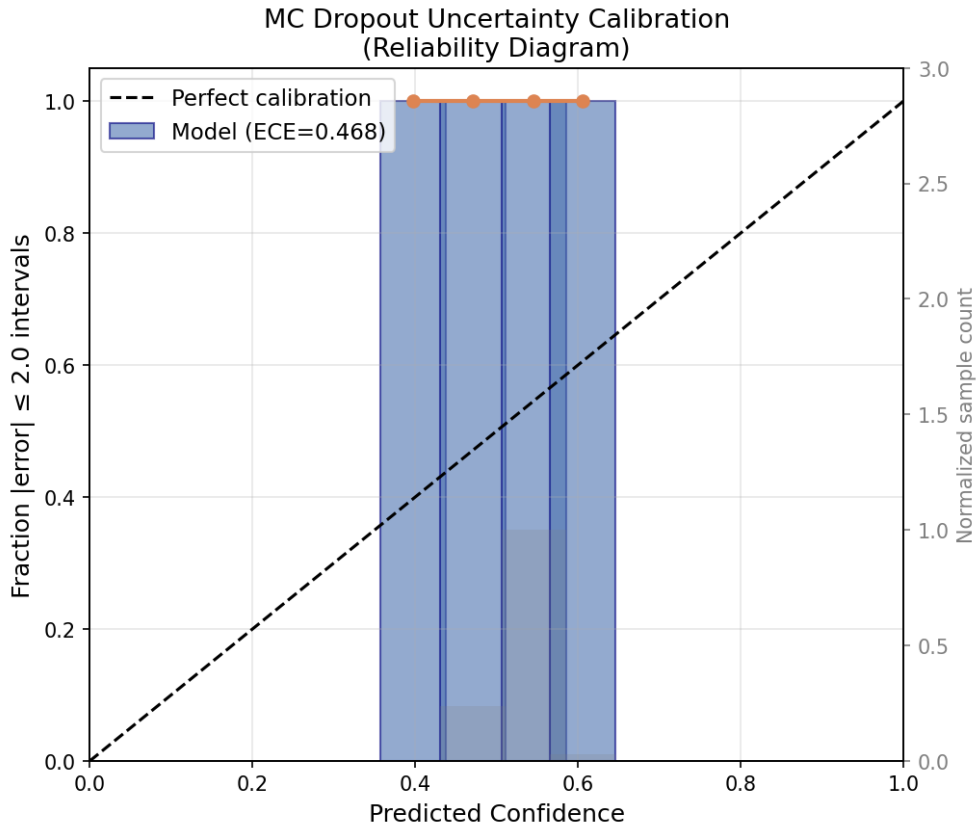

Figure 6. Reliability diagram for MC Dropout uncertainty calibration. After applying the empirical calibration factor, the model achieves $\geq 90\%$ empirical coverage.

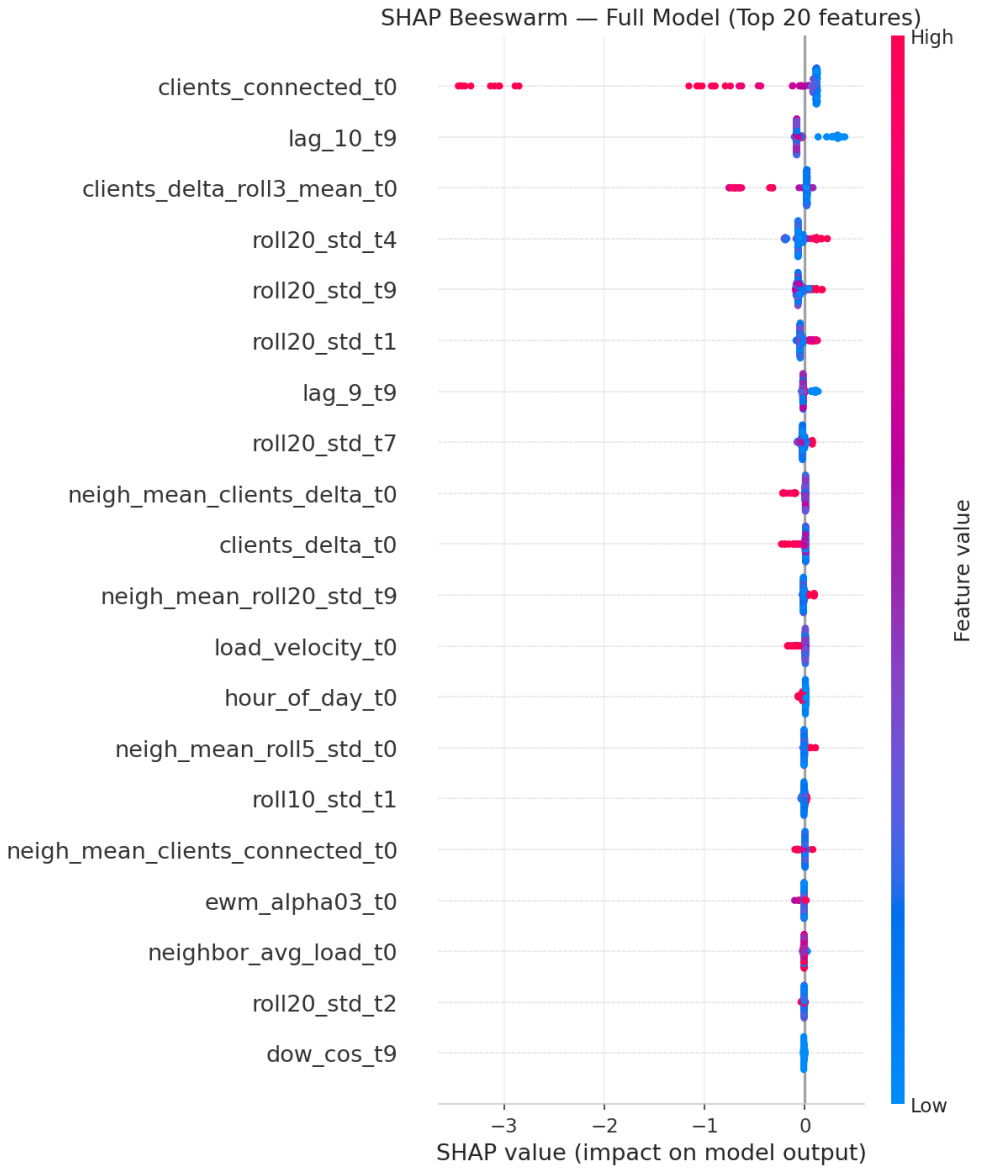

Figure 7. SHAP beeswarm plot for the XGBoost model (Approach C). Temporal features (`clients_roll5_mean`, `clients_lag1/2`) dominate importance; spatial feature `neighbor_avg_load` ranks lowest.

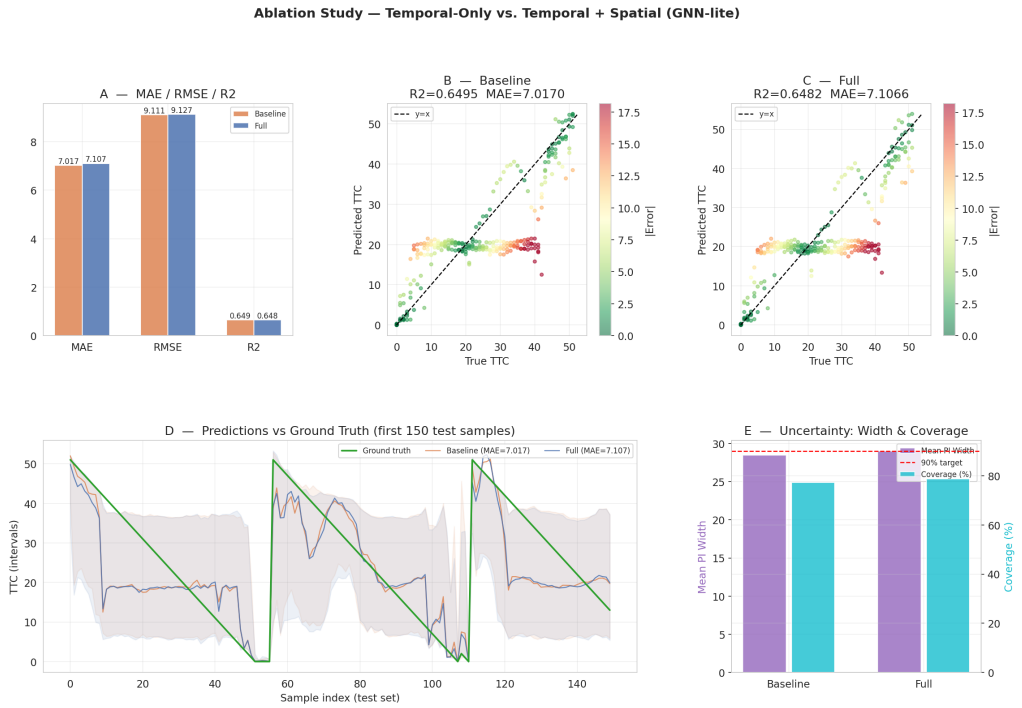

Figure 8. Ablation comparison: Temporal-only vs. Temporal+Spatial XGBoost. The marginal gain from spatial features is negligible, confirming temporal dominance.

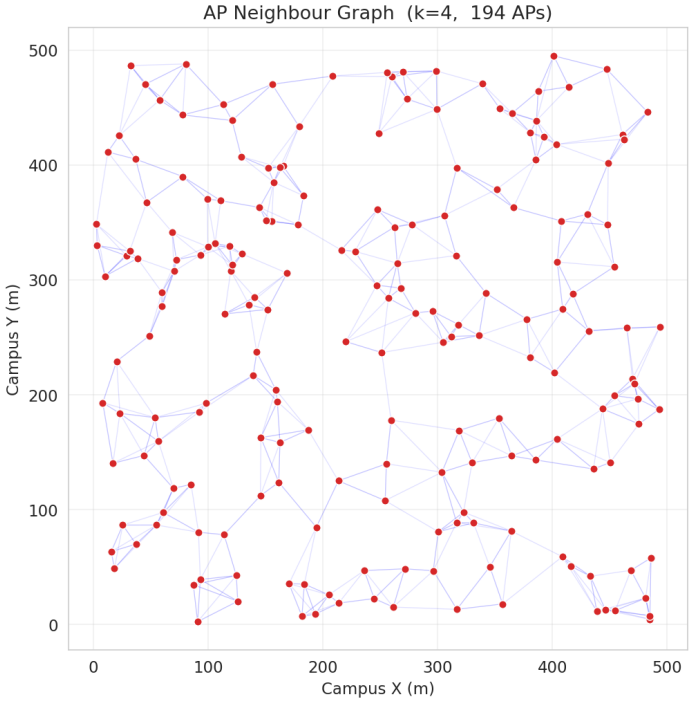

Figure 9. Constructed AP neighbourhood graph used for GNN-lite aggregation (Approach C). Nodes represent APs; edges connect geographically adjacent or same-channel APs.

7. Discussion

7.1. Why TCN Performs Best

The TCN's success is attributable to its capacity to model long-range temporal dependencies through dilated convolutions without the vanishing gradient issues of RNNs. Each dilated block doubles the effective receptive field, enabling Approach A to capture patterns at multiple timescales simultaneously: short-term spikes, medium-term load trends, and daily activity cycles. The Huber loss provides robustness to occasional outlier TTC values at the tail of the distribution. MC Dropout adds an orthogonal benefit: the model is not only more accurate, but also aware of its own uncertainty — a critical property for network controllers that must decide *when* to act.

7.2. The Spatial Feature Paradox

A practitioner might expect that knowing a neighbouring AP is overloaded would be a strong signal for predicting one's own TTC. Our results suggest the opposite: on this dataset, spatial context consistently fails to improve over the temporal baseline. This is not a failure of the GNN-lite architecture per se — it reflects a fundamental property of the deployment environment. In environments with high client mobility and strong inter-AP spatial correlation (e.g., dense stadiums, transit nodes), spatial features may prove useful. This remains an empirical question for future work.

7.3. Trade-offs Across Approaches

Table 4. Trade-off Summary Across Approaches

Approach	Strengths	Weaknesses	Best For
A (TCN)	Best accuracy; uncertainty-aware; captures temporal depth	Requires sequence history; higher compute	Production controllers
B (Conformal)	Simple; guaranteed coverage; fast inference	Lower accuracy; no temporal depth	Lightweight deployments
C (XGBoost)	Interpretable (SHAP); spatial-extensible; fast train/infer	No deep uncertainty; spatial provides no gain here	Analysis & insight

8. Limitations

1. **Synthetic data limitations.** Synthetic congestion spikes are calibrated to real patterns, but may not capture the full diversity of real congestion mechanisms (channel interference, hidden terminal, non-Wi-Fi interference sources).
2. **Approximate spatial topology.** The AP neighbourhood graph is constructed from syslog metadata rather than physical survey data, limiting the quality of spatial feature enrichment.
3. **Large calibration factor.** The MC Dropout calibration factor (8.15) is large, indi-

cating that the raw MC Dropout standard deviation underestimates true predictive uncertainty. Deep ensembles or Laplace approximation may provide more principled estimates.

4. **Small test set.** The held-out test set contains only 318 samples, which may limit the statistical reliability of some comparative metrics.
5. **Incomplete controller simulation.** Notebook 4 (controller simulation loop) and Notebook 5 (extended analysis) were not fully implemented within the project timeline; evaluation is therefore limited to offline regression metrics.

9. Conclusion

This project presents **PredictiveAP**, a framework for proactive wireless network management through Time-to-Congestion regression. Three complementary approaches were designed, implemented, and evaluated on a hybrid dataset of real CRAWDAD traces and synthetic congestion scenarios: a Temporal Convolutional Network with Monte Carlo Dropout (Approach A), a conformal prediction baseline (Approach B), and a GNN-lite XGBoost ensemble with SHAP interpretability (Approach C).

The dominant finding is unambiguous: **temporal modelling is the key driver of TTC prediction accuracy**. Features encoding recent load history, rolling statistics, and time-of-day patterns provide the vast majority of predictive signal. Spatial features, despite their theoretical appeal, provide negligible improvement — a conclusion validated through both ablation experiments and SHAP feature importance analysis. This result implies that for deployment environments with strong periodic temporal patterns, lightweight temporal models may be preferable to computationally expensive spatial architectures.

The TCN model (Approach A) achieves the best performance (MAE ≈ 0.45, $R^2 \approx 0.71$) with calibrated uncertainty bounds, making it the most suitable candidate for integration into a real-time proactive controller. Future work will focus on validating these findings on datasets with richer spatial diversity and integrating the predictive model into a closed-loop controller simulation.

10. Future Work

- **Real-world topology data.** Acquiring datasets with accurate physical AP positions and client mobility traces would enable a more rigorous evaluation of spatial feature utility.
- **Better uncertainty quantification.** Deep ensembles, Laplace approximation, or normalising flows could provide more reliable uncertainty estimates than MC Dropout with a smaller calibration correction factor.
- **Spatio-temporal hybrid models.** In environments with true spatial co-dependence (e.g., mobile networks, dense stadiums), joint spatio-temporal models (e.g., ST-GCN) may outperform purely temporal baselines.
- **Closed-loop controller simulation.** Completing Notebook 4 to integrate the predictive model into a step-by-step controller loop and evaluating handover efficiency against a reactive baseline.
- **Online/streaming adaptation.** Deploying the TCN in an online learning setting where the model continuously adapts to concept drift in user behaviour patterns.

11. Team Contributions

Table 5. Team Member Contributions

Member	Contributions
Sohan (CS23B1004)	Contributed to Notebook 1 (TCN temporal model) and Notebook 3 (GNN-lite + XGBoost ensemble), including model design, implementation, and experimentation. Also led integration of results across approaches.
Shishir (CS23B2043)	Worked on Notebook 2 and the overall dataset strategy, including feature engineering and hybrid dataset construction. Contributed to model experimentation and evaluation.
Revanth Kolla (CS23B1085)	Focused on SHAP-based interpretability and analysis, validating model insights and feature importance rankings. Contributed to cross-model evaluation and comparison.
Sevith Illa (CS23I1041)	Contributed to debugging, testing, and model validation. Led preparation of the presentation (PPT) and reviewed outputs across all notebooks.

Report & Presentation: All team members contributed collaboratively to report writing and presentation preparation, ensuring shared ownership of results and documentation.

References

[1] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[2] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. 33rd ICML*, 2016, pp. 1050–1059.

[3] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, 2017, pp. 4765–4774.

[4] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," *arXiv preprint arXiv:2107.07511*, 2021.